

Month 3 Assessment

Assessment Scenario

You're now a senior DevOps engineer at StartTech, and the company is ready to implement a complete CI/CD pipeline for their full-stack application. The application has grown to include:

- **Frontend:** React application (to be deployed to AWS S3)
- **Backend API:** Golang application (to be deployed to EC2 with autoscaling)
- **Redis:** ElastiCache cluster for caching and sessions
- **Database:** MongoDB for data persistence(Create one on Mongo Atlas).

Your mission is to create a comprehensive CI/CD pipeline that automates the entire deployment process from code commit to production, including proper monitoring and security practices.

Application Architecture

The application [[repo](#)] now consists of:

- **Frontend (React):** Static files served from S3 with CloudFront CDN
- **Backend API:** Golang application running on EC2 instances behind ALB
- **Redis:** ElastiCache cluster for caching and sessions
- **MongoDB:** MongoDB Atlas or EC2-hosted MongoDB
- **Infrastructure:** All managed with Terraform

Technical Requirements

Phase 1: Infrastructure as Code

Update your Terraform configuration to include:

- **Auto Scaling Group** for backend EC2 instances
- **Application Load Balancer** with target group
- **S3 bucket** for frontend hosting with static website configuration
- **CloudFront distribution** for global content delivery
- **ElastiCache Redis cluster** for caching
- **CloudWatch Log Groups** for application logging
- **IAM roles and policies** for EC2 instances to access CloudWatch
- **Security Groups** for all components

Phase 2: CI/CD Pipeline Development

Frontend Pipeline (React to S3)

Create GitHub Actions workflow that:

1. **Build Stage:**
 - Installs Node.js dependencies
 - Runs unit tests
 - Builds production-ready React bundle
 - Runs security scanning (npm audit)
2. **Deploy Stage:**
 - Syncs build files to S3 bucket
 - Invalidates CloudFront cache
 - Sends deployment notifications

Backend Pipeline (Golang to EC2)

Create GitHub Actions workflow that:

1. **Test Stage:**
 - Runs unit tests and integration tests
 - Code quality checks
 - Vulnerability scanning
2. **Build Stage:**
 - Builds Docker image
 - Scans Docker image for vulnerabilities
 - Tags and pushes to ECR/Docker Hub
3. **Deploy Stage:**
 - Runs smoke tests
 - Deploy using rolling update
 - Configures auto-scaling policies
 - Sets up CloudWatch monitoring collecting logs into cloudwatch log group.

Phase 3: Monitoring and Observability

Application Monitoring

- **CloudWatch Logs:** Centralized logging for all services

Submission Requirements

Required Repository Structure:

None

```
starttech-infra/
├── .github/
│   └── workflows/
│       └── infrastructure-deploy.yml
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   ├── outputs.tf
│   ├── modules/
│   │   ├── networking/
│   │   ├── compute/
│   │   ├── storage/
│   │   └── monitoring/
│   └── terraform.tfvars.example
├── scripts/
│   └── deploy-infrastructure.sh
├── monitoring/
│   ├── cloudwatch-dashboard.json
│   ├── alarm-definitions.json
│   └── log-insights-queries.txt
└── README.md
```

None

```
starttech-application/
├── .github/
│   └── workflows/
│       ├── frontend-ci-cd.yml
│       └── backend-ci-cd.yml
├── frontend/
├── backend/
├── scripts/
│   ├── deploy-frontend.sh
│   ├── deploy-backend.sh
│   ├── health-check.sh
│   └── rollback.sh
```

1. CI/CD Pipelines

- **Frontend Pipeline:** Complete React build and S3 deployment
- **Backend Pipeline:** Complete Docker build and EC2 deployment
- **Infrastructure Pipeline:** Terraform deployment automation

2. Infrastructure Code

- **Terraform Modules:** Organized, reusable infrastructure modules
- **Auto Scaling:** Configured ASG with proper scaling policies
- **Load Balancing:** ALB with health checks and target groups
- **Monitoring:** CloudWatch integration for logs and metrics

3. Application Code

- **Frontend:** React application with environment-specific configurations
- **Backend:** Golang API with health endpoints and logging
- **Configuration:** Proper environment variable management

4. Monitoring Setup

- **Log Analysis:** Structured logging with CloudWatch Logs Insights

5. Documentation

- **README.md:** Comprehensive setup and deployment guide
- **ARCHITECTURE.md:** System architecture documentation
- **RUNBOOK.md:** Operations and troubleshooting guide

6. Security Implementation

- **Secrets Management:** Proper handling of API keys and credentials
- **Security Scanning:** Automated vulnerability scanning in pipelines
- **IAM Policies:** Least-privilege access controls
- **Network Security:** Proper security group configurations

Submission Method:

1. Fork the application repository, where you would make modifications setting up a CI/CD pipeline with github actions.
2. Create a different repository for infrastructure setup, with its own CI/CD pipeline.
3. Implement all required components
4. Create comprehensive documentation
5. Submit both repository URLs, along with AWS console and CLI credentials. This user credential **MUST** have the necessary IAM policies set up to allow access to the AWS resources involved in this assessment.